

Medicine Availability Check & Pharmacy Classification by Distance

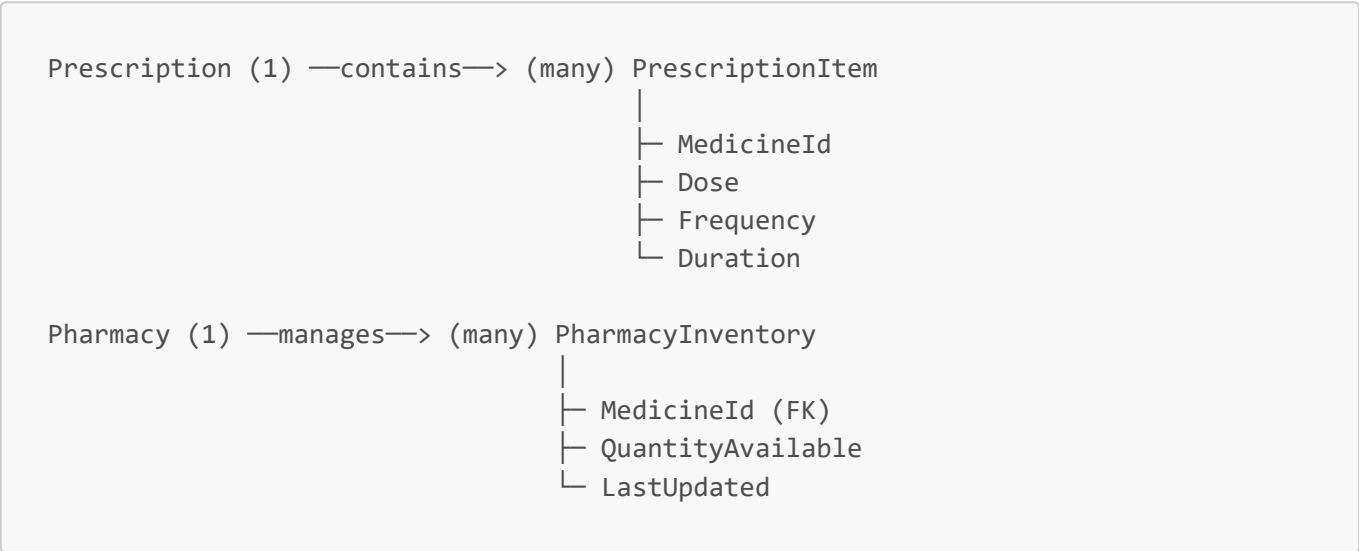
Overview

This document explains how the system:

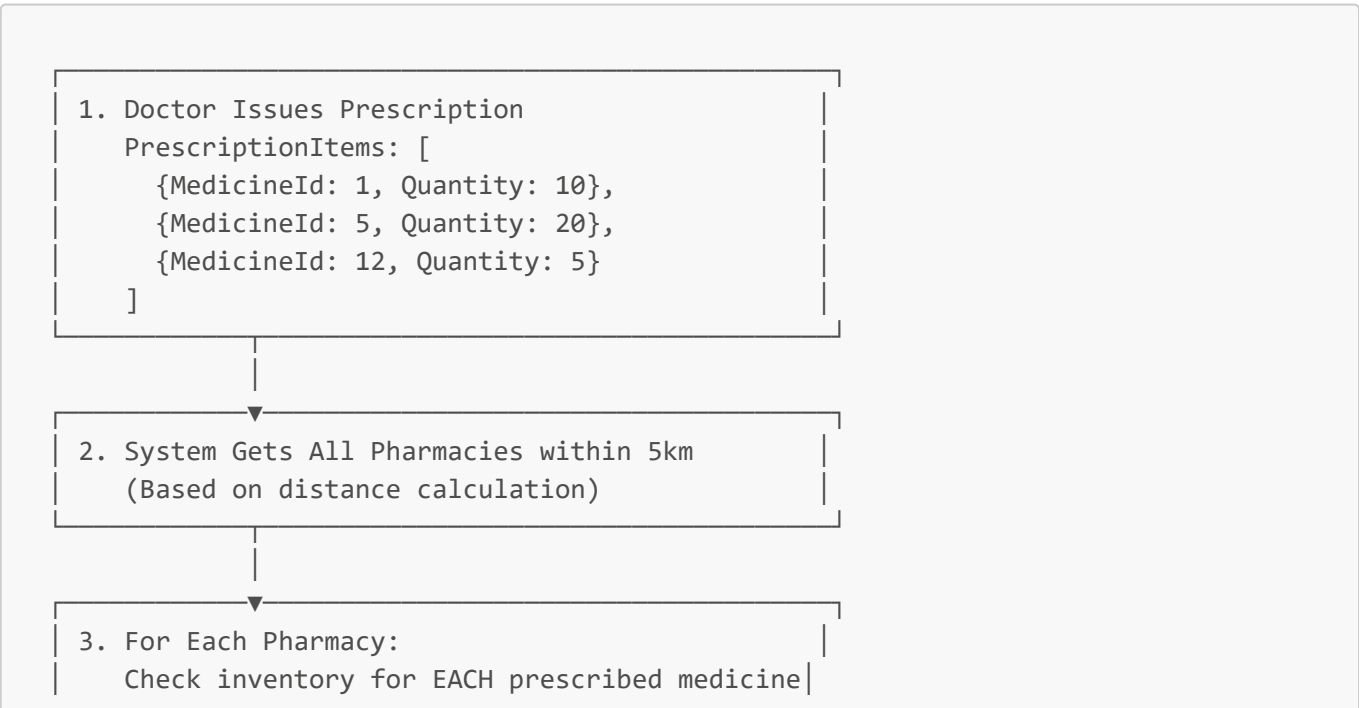
- 1. **Checks medicine availability** at each pharmacy based on prescription requirements
- 2. **Classifies pharmacies** and ranks them by distance while considering stock availability

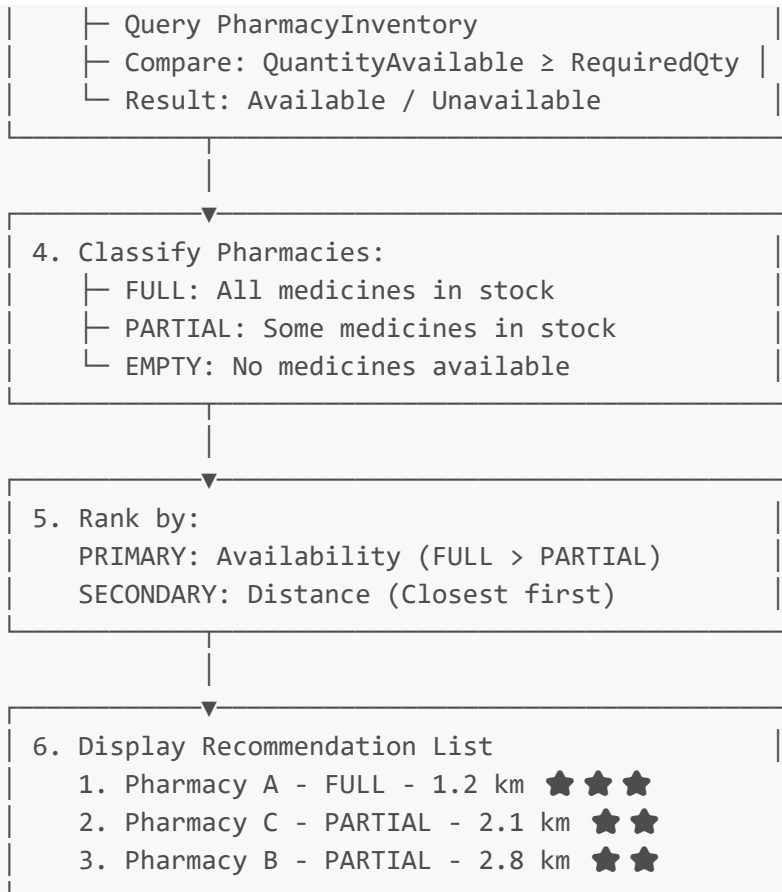
Part 1: Medicine Availability Check Flow

Data Model Reference



Process Flow





Part 2: SQL Queries for Availability Check

Query 1: Get Prescription Items for a Prescription

```

-- Get all medicines required in a prescription
SELECT
    pi.PrescriptionItemId,
    pi.MedicineId,
    pi.Name AS MedicineName,
    pi.Dose,
    pi.Frequency,
    pi.Duration,
    -- Calculate total quantity needed
    CASE
        WHEN pi.Frequency = 'Once Daily' THEN DATEDIFF(DAY, GETDATE(),
DATEADD(DAY, 30, GETDATE()))
        WHEN pi.Frequency = 'Twice Daily' THEN DATEDIFF(DAY, GETDATE(),
DATEADD(DAY, 30, GETDATE())) * 2
        WHEN pi.Frequency = 'Three Times Daily' THEN DATEDIFF(DAY, GETDATE(),
DATEADD(DAY, 30, GETDATE())) * 3
        ELSE 1
    END AS RequiredQuantity
FROM PrescriptionItem pi
WHERE pi.PrescriptionId = @prescriptionId
ORDER BY pi.PrescriptionItemId;
    
```

Query 2: Check Pharmacy Stock for All Medicines in Prescription

```
-- Check which pharmacies have ALL prescribed medicines in stock
SELECT
    p.PharmacyId,
    p.Name AS PharmacyName,
    a.FormattedAddress,
    a.Wilaya,
    a.Commune,
    -- Calculate distance from doctor's clinic
    dbo.CalculateDistance(@doctorLat, @doctorLng, a.Lat, a.Lng) AS DistanceKm,
    -- Count available items
    COUNT(DISTINCT CASE
        WHEN pi.MedicineId = inv.MedicineId AND inv.QuantityAvailable >=
pi.RequiredQuantity
        THEN pi.PrescriptionItemId
    END) AS AvailableItemsCount,
    -- Total items in prescription
    (SELECT COUNT(*) FROM PrescriptionItem WHERE PrescriptionId = @prescriptionId)
AS TotalItemsCount
FROM Pharmacy p
    INNER JOIN Address a ON p.AddressId = a.AddressId
    LEFT JOIN PharmacyInventory inv ON p.PharmacyId = inv.PharmacyId
    CROSS JOIN (
        SELECT pi.MedicineId,
            -- Calculate required quantity
            CASE
                WHEN pi.Frequency = 'Once Daily' THEN 30
                WHEN pi.Frequency = 'Twice Daily' THEN 60
                ELSE 1
            END AS RequiredQuantity
        FROM PrescriptionItem pi
        WHERE pi.PrescriptionId = @prescriptionId
    ) pi
WHERE
    p.IsActive = 1
    AND dbo.CalculateDistance(@doctorLat, @doctorLng, a.Lat, a.Lng) <= 5.0
GROUP BY
    p.PharmacyId, p.Name, a.FormattedAddress, a.Wilaya, a.Commune, a.Lat, a.Lng
ORDER BY
    -- First: FULL availability (all items available)
    CASE
        WHEN COUNT(DISTINCT CASE WHEN pi.MedicineId = inv.MedicineId THEN
pi.PrescriptionItemId END) =
            (SELECT COUNT(*) FROM PrescriptionItem WHERE PrescriptionId =
@prescriptionId)
        THEN 1
        ELSE 2
    END ASC,
    -- Second: Distance (closest first)
    DistanceKm ASC;
```

Query 3: Detailed Availability Report Per Pharmacy

```
-- Get detailed breakdown: which medicines available/unavailable at each pharmacy
SELECT
    p.PharmacyId,
    p.Name AS PharmacyName,
    a.FormattedAddress,
    dbo.CalculateDistance(@doctorLat, @doctorLng, a.Lat, a.Lng) AS DistanceKm,
    pi.MedicineId,
    pi.Name AS MedicineName,
    pi.Dose,
    pi.Frequency,
    CASE
        WHEN pi.Frequency = 'Once Daily' THEN 30
        WHEN pi.Frequency = 'Twice Daily' THEN 60
        WHEN pi.Frequency = 'Three Times Daily' THEN 90
        ELSE 1
    END AS RequiredQuantity,
    COALESCE(inv.QuantityAvailable, 0) AS AvailableQuantity,
    CASE
        WHEN COALESCE(inv.QuantityAvailable, 0) >=
            CASE
                WHEN pi.Frequency = 'Once Daily' THEN 30
                WHEN pi.Frequency = 'Twice Daily' THEN 60
                ELSE 1
            END
        THEN 'AVAILABLE'
        ELSE 'UNAVAILABLE'
    END AS AvailabilityStatus
FROM Pharmacy p
    INNER JOIN Address a ON p.AddressId = a.AddressId
    INNER JOIN PrescriptionItem pi ON pi.PrescriptionId = @prescriptionId
    LEFT JOIN PharmacyInventory inv ON p.PharmacyId = inv.PharmacyId
        AND pi.MedicineId = inv.MedicineId
WHERE
    p.IsActive = 1
    AND dbo.CalculateDistance(@doctorLat, @doctorLng, a.Lat, a.Lng) <= 5.0
ORDER BY
    p.PharmacyId,
    pi.MedicineId;
```

Part 3: Pharmacy Classification System

Classification Categories

Category 1: Availability Level

CLASSIFICATION BY MEDICINE AVAILABILITY

- FULL (100%)
 - All prescribed medicines in stock
 - ✓ Best choice
 - ✓ Patient gets everything same day
- PARTIAL (50-99%)
 - Some medicines available
 - ⚠ Patient must wait for others
 - ⚠ May need to visit multiple pharmacies
- EMPTY (0%)
 - No medicines available
 - ✗ Not recommended
 - ✗ Patient needs different pharmacy

Classification Algorithm (Pseudocode)

```
def classify_pharmacy(pharmacy_id, prescription_id, prescription_items):
    """
    Classify pharmacy based on medicine availability
    Returns: "FULL", "PARTIAL", or "EMPTY"
    """

    total_items = len(prescription_items)
    available_count = 0

    for item in prescription_items:
        required_qty = calculate_required_quantity(item.frequency)

        inventory = query_inventory(
            pharmacy_id=pharmacy_id,
            medicine_id=item.medicine_id
        )

        if inventory and inventory.quantity_available >= required_qty:
            available_count += 1

    # Calculate percentage
    availability_percentage = (available_count / total_items) * 100

    # Classify
    if availability_percentage == 100:
        return "FULL"
    elif availability_percentage > 0:
```

```

        return "PARTIAL"
    else:
        return "EMPTY"

```

Ranking Algorithm

```

def rank_pharmacies(prescriptions_items, doctor_location, max_distance_km=5):
    """
    Rank pharmacies by availability first, then distance
    Returns: Sorted list of pharmacies
    """

    pharmacies_with_scores = []

    # Get all pharmacies within range
    nearby_pharmacies = get_pharmacies_within_distance(doctor_location,
max_distance_km)

    for pharmacy in nearby_pharmacies:
        # Step 1: Classify by availability
        availability = classify_pharmacy(pharmacy.id, prescriptions_items)

        # Step 2: Calculate distance
        distance = calculate_haversine_distance(
            doctor_location.lat, doctor_location.lng,
            pharmacy.address.lat, pharmacy.address.lng
        )

        # Step 3: Create ranking score
        score = {
            'pharmacy': pharmacy,
            'availability': availability,
            'distance': distance,
            'rank_priority': get_priority(availability) # FULL=1, PARTIAL=2,
EMPTY=3
        }

        pharmacies_with_scores.append(score)

    # Step 4: Sort by priority, then distance
    pharmacies_with_scores.sort(
        key=lambda x: (x['rank_priority'], x['distance'])
    )

    return pharmacies_with_scores

def get_priority(availability_class):
    """Map availability to priority number"""
    priorities = {
        'FULL': 1,      # Highest priority
        'PARTIAL': 2,

```

```
'EMPTY': 3      # Lowest priority
}
return priorities.get(availability_class, 4)
```

Part 4: Example Scenario with Availability

Real Scenario: Patient's Prescription

Doctor: Dr. Hassan

Clinic Location: Algiers (36.7538° N, 3.0588° E)

Prescription Items:

- Medicine A: Aspirin 500mg → Qty: 30 (Once Daily × 30 days)
- Medicine B: Vitamin C → Qty: 30 (Once Daily × 30 days)
- Medicine C: Antibiotic → Qty: 60 (Twice Daily × 30 days)

Available Pharmacies within 5km:

- Pharmacy A (Ben Aknoun) - 1.2 km
- Pharmacy B (Kouba) - 2.8 km
- Pharmacy C (Mustafa Pasha) - 2.1 km
- Pharmacy D (Riad) - 3.5 km

Inventory Check Results

PHARMACY A - BEN AKNOUN (1.2 km)			
Medicine A (Aspirin)	Need: 30	Have: 40	✓ AVAILABLE
Medicine B (Vitamin C)	Need: 30	Have: 25	✗ UNAVAILABLE
Medicine C (Antibiotic)	Need: 60	Have: 75	✓ AVAILABLE
Classification:  PARTIAL (2/3 medicines available = 66%)			

PHARMACY B - KOUBA (2.8 km)			
Medicine A (Aspirin)	Need: 30	Have: 10	✗ UNAVAILABLE
Medicine B (Vitamin C)	Need: 30	Have: 20	✗ UNAVAILABLE
Medicine C (Antibiotic)	Need: 60	Have: 0	✗ UNAVAILABLE
Classification:  EMPTY (0/3 medicines available = 0%)			

PHARMACY C - MUSTAFA PASHA (2.1 km)			
Medicine A (Aspirin)	Need: 30	Have: 50	✓ AVAILABLE

Medicine B (Vitamin C)	Need: 30	Have: 35	✓ AVAILABLE
Medicine C (Antibiotic)	Need: 60	Have: 80	✓ AVAILABLE
Classification: ● FULL (3/3 medicines available = 100%)			

PHARMACY D - RIAD (3.5 km)			
Medicine A (Aspirin)	Need: 30	Have: 50	✓ AVAILABLE
Medicine B (Vitamin C)	Need: 30	Have: 30	✓ AVAILABLE
Medicine C (Antibiotic)	Need: 60	Have: 15	✗ UNAVAILABLE
Classification: ● PARTIAL (2/3 medicines available = 66%)			

Ranked Recommendation List

- FINAL RANKING (by availability, then distance):
- 1. ● PHARMACY C - MUSTAFA PASHA
 - Distance: 2.1 km
 - Availability: FULL (100%)
 - ★★★ BEST CHOICE
 - └ All medicines available - Get everything today!
 - 2. ● PHARMACY A - BEN AKNOUN
 - Distance: 1.2 km
 - Availability: PARTIAL (66%)
 - ★★ (Closer, but missing Vitamin C)
 - └ Missing: Vitamin C
 - 3. ● PHARMACY D - RIAD
 - Distance: 3.5 km
 - Availability: PARTIAL (66%)
 - ★★ (Farther away)
 - └ Missing: Antibiotic
 - 4. ● PHARMACY B - KOUBA
 - Distance: 2.8 km
 - Availability: EMPTY (0%)
 - ✗ NOT RECOMMENDED
 - └ No medicines in stock

Part 5: Implementation in C# (.NET)

Model Classes

```

public class PharmacyRecommendation
{
    public int PharmacyId { get; set; }
    public string Name { get; set; }
    public string FormattedAddress { get; set; }
    public decimal DistanceKm { get; set; }
    public AvailabilityLevel Availability { get; set; }
    public decimal AvailabilityPercentage { get; set; }
    public List<MedicineAvailability> MedicineDetails { get; set; }
    public int Rank { get; set; }
}

public enum AvailabilityLevel
{
    Full = 1,           // All medicines available
    Partial = 2,        // Some medicines available
    Empty = 3           // No medicines available
}

public class MedicineAvailability
{
    public int MedicineId { get; set; }
    public string MedicineName { get; set; }
    public int RequiredQuantity { get; set; }
    public int AvailableQuantity { get; set; }
    public bool IsAvailable { get; set; }
}

```

Service Method

```

public class PharmacyRecommendationService
{
    private readonly IPharmacyRepository _pharmacyRepository;
    private readonly IInventoryRepository _inventoryRepository;
    private readonly IDistanceService _distanceService;

    public async Task<List<PharmacyRecommendation>> GetRecommendedPharmacies(
        int prescriptionId,
        decimal doctorLat,
        decimal doctorLng,
        decimal maxDistanceKm = 5.0m)
    {
        // Step 1: Get prescription items
        var prescriptionItems = await
        _pharmacyRepository.GetPrescriptionItems(prescriptionId);

        // Step 2: Get nearby pharmacies
        var nearbyPharmacies = await
        _pharmacyRepository.GetPharmaciesWithinDistance(
            doctorLat, doctorLng, maxDistanceKm);
    }
}

```

```
var recommendations = new List<PharmacyRecommendation>();

// Step 3: For each pharmacy, check availability
foreach (var pharmacy in nearbyPharmacies)
{
    var medicineAvailability = await CheckMedicineAvailability(
        pharmacy.PharmacyId,
        prescriptionItems);

    var availabilityLevel = ClassifyAvailability(medicineAvailability);
    var distance = _distanceService.CalculateHaversine(
        doctorLat, doctorLng,
        pharmacy.Address.Lat, pharmacy.Address.Lng);

    recommendations.Add(new PharmacyRecommendation
    {
        PharmacyId = pharmacy.PharmacyId,
        Name = pharmacy.Name,
        FormattedAddress = pharmacy.Address.FormattedAddress,
        DistanceKm = distance,
        Availability = availabilityLevel,
        AvailabilityPercentage = (decimal)medicineAvailability.Count(m =>
m.IsAvailable)
            / medicineAvailability.Count * 100,
        MedicineDetails = medicineAvailability
    });
}

// Step 4: Sort by availability (FULL first), then by distance
return recommendations
    .OrderBy(r => r.Availability)
    .ThenBy(r => r.DistanceKm)
    .Select((r, index) => { r.Rank = index + 1; return r; })
    .ToList();
}

private async Task<List<MedicineAvailability>> CheckMedicineAvailability(
    int pharmacyId,
    List<PrescriptionItem> items)
{
    var availabilityList = new List<MedicineAvailability>();

    foreach (var item in items)
    {
        var requiredQty = CalculateRequiredQuantity(item.Frequency);
        var inventory = await _inventoryRepository.GetInventory(
            pharmacyId, item.MedicineId);

        var available = inventory != null && inventory.QuantityAvailable >=
requiredQty;

        availabilityList.Add(new MedicineAvailability
        {
```

```

        MedicineId = item.MedicineId,
        MedicineName = item.Name,
        RequiredQuantity = requiredQty,
        AvailableQuantity = inventory?.QuantityAvailable ?? 0,
        IsAvailable = available
    });
}

return availabilityList;
}

private AvailabilityLevel ClassifyAvailability(List<MedicineAvailability>
medicines)
{
    if (medicines.All(m => m.IsAvailable))
        return AvailabilityLevel.Full;

    if (medicines.Any(m => m.IsAvailable))
        return AvailabilityLevel.Partial;

    return AvailabilityLevel.Empty;
}

private int CalculateRequiredQuantity(string frequency)
{
    return frequency switch
    {
        "Once Daily" => 30,
        "Twice Daily" => 60,
        "Three Times Daily" => 90,
        _ => 1
    };
}
}

```

Part 6: Database Schema

Updated Schema with Inventory

```

-- Pharmacy Inventory table
CREATE TABLE PharmacyInventory (
    PharmacyInventoryId INT PRIMARY KEY IDENTITY,
    PharmacyId INT NOT NULL FOREIGN KEY REFERENCES Pharmacy(PharmacyId),
    MedicineId INT NOT NULL FOREIGN KEY REFERENCES Medicine(MedicineId),
    QuantityAvailable INT NOT NULL DEFAULT 0,
    MinimumThreshold INT NOT NULL DEFAULT 10, -- Reorder point
    LastUpdated DATETIME DEFAULT GETDATE(),
    LastRestocked DATETIME,
    CONSTRAINT UC_Pharmacy_Medicine UNIQUE (PharmacyId, MedicineId)
);

```

```
-- Prescription Item table (links medicines to prescriptions)
CREATE TABLE PrescriptionItem (
  PrescriptionItemId INT PRIMARY KEY IDENTITY,
  PrescriptionId INT NOT NULL FOREIGN KEY REFERENCES
Prescription(PrescriptionId),
  MedicineId INT NOT NULL FOREIGN KEY REFERENCES Medicine(MedicineId),
  Name NVARCHAR(255) NOT NULL,
  Dose NVARCHAR(100),
  Frequency NVARCHAR(50), -- 'Once Daily', 'Twice Daily', etc.
  Duration INT, -- Number of days
  Type NVARCHAR(50), -- 'Medication', 'OxygenTherapy', etc.
  Notes NVARCHAR(500)
);

-- Create index for fast pharmacy inventory lookups
CREATE INDEX IX_PharmacyInventory_Pharmacy_Medicine
ON PharmacyInventory(PharmacyId, MedicineId);

-- Create index for fast quantity checks
CREATE INDEX IX_PharmacyInventory_Quantity
ON PharmacyInventory(PharmacyId, QuantityAvailable);
```

Part 7: User Interface Flow

Patient View

PRESCRIPTION RECOMMENDATION SCREEN

Your Prescription:

└ Aspirin 500mg (Once Daily, 30 days) × 30 tablets

└ Vitamin C (Once Daily, 30 days) × 30 tablets

└ Antibiotic (Twice Daily, 30 days) × 60 tablets

RECOMMENDED PHARMACIES:

[1] PHARMACY C - MUSTAFA PASHA [SELECT]

2.1 km away

✓ All medicines in stock (100%)

★ ★ ★ BEST OPTION

[2] PHARMACY A - BEN AKNOUN [SELECT]

1.2 km away

⚠ 2 of 3 medicines in stock (66%)

Missing: Vitamin C

★ ★

[3]

PHARMACY D - RIAD

[SELECT]

3.5 km away

2 of 3 medicines in stock (66%)

Missing: Antibiotic

[4]

PHARMACY B - KOUBA

[UNAVAILABLE]

2.8 km away

No medicines available (0%)

Not recommended

Part 8: Edge Cases & Considerations

1. Insufficient Quantity at All Pharmacies

Scenario: Patient needs 100 tablets, max 30 available anywhere

Solution:

- Rank by highest available quantity first
- Show: "25/100 tablets available at this pharmacy"
- Suggestion: "Contact pharmacy to special order"

2. Time-Sensitive Medicines

Scenario: Antibiotic must start within 24 hours

Solution:

- Flag:  "Time-sensitive - select pharmacy NOW"
- Highlight: Only pharmacies with full availability
- Feature: Skip partial matches

3. Pharmacy Hours Consideration

Scenario: Closest pharmacy closes in 2 hours

Solution:

- Query: WHERE IsOpen = 1 AND ClosingTime > NOW
- Show: " Closes in 1 hour 45 minutes"
- Rank: Open pharmacies first

4. Stock Updates in Real-Time

Problem: Inventory changes between query and selection

Solution:

- | Auto-refresh inventory every 30 seconds
- | Show: "Last updated 2 minutes ago"
- | Reserve: Immediately reserve items when patient selects
- | Lock: Prevent double-booking

Part 9: Performance Optimization

Caching Strategy

```
// Cache pharmacy availability for 30 minutes
var cacheKey = $"PharmacyRecs_{prescriptionId}";
if (!_cache.TryGetValue(cacheKey, out var recommendations))
{
    recommendations = await GetRecommendedPharmacies(prescriptionId, doctorLat,
doctorLng);

    _cache.Set(cacheKey, recommendations,
        new MemoryCacheEntryOptions
        {
            AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(30)
        });
}
```

Database Indexes

```
-- Speed up pharmacy lookup by distance
CREATE SPATIAL INDEX IX_Address_Spatial
ON Address(Lat, Lng);

-- Speed up inventory lookups
CREATE INDEX IX_PharmacyInventory_Pharmacy
ON PharmacyInventory(PharmacyId, QuantityAvailable);

-- Speed up medicine availability checks
CREATE INDEX IX_PrescriptionItem_Medicine
ON PrescriptionItem(PrescriptionId, MedicineId);
```

Part 10: Notification System

Stock Alert to Patient

When pharmacy is selected:

NOTIFICATION
Pharmacy confirmed availability for your prescription:
✓ Aspirin - 40 tablets (need 30)
✓ Vitamin C - 35 tablets (need 30)
✓ Antibiotic - 80 tablets (need 60)
Ready for pickup in 30 minutes
Location: 2.1 km away
[PICKUP CONFIRMATION] [CHANGE PHARMACY]

Summary

Medicine Availability System:

- ✓ Checks PharmacyInventory for each medicine in prescription
- ✓ Compares required quantity vs available quantity
- ✓ Classifies pharmacies: FULL (100%), PARTIAL (50-99%), EMPTY (0%)

Pharmacy Classification & Ranking:

- PRIMARY: Availability level (FULL > PARTIAL > EMPTY)
- SECONDARY: Distance to pharmacy (closest first)
- RESULT: Optimal pharmacy choice for patient

Benefits:

- Faster pharmacy selection
- Better user experience
- Reduced wasted trips
- Fewer unfulfilled prescriptions